TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

CƠ SỞ TRÍ TUỆ NHÂN TẠO

FUTOSHIKI

Giải bài toán bằng Logic vị từ bậc nhất và các thuật toán suy diễn

Giảng viên hướng dẫn: Thầy Lê Ngọc Thành
Thầy Huỳnh Lâm Hải Đăng
Thầy Lê Nhựt Nam

Sinh viên thực hiện:

Họ và tên	Mã số sinh viên	Lớp
Nguyễn Nhựt Huy	24127398	24C11
Trần Quang Minh Huy	24127403	24C11
Trịnh Khánh Linh	24127437	24C11
Nguyễn Minh Khôi	24127066	24C11

TP. Hồ Chí Minh, tháng 2/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

THÔNG TIN VỀ ĐỒ ÁN

Mã học phần: CSC14003

Tên học phần: Cơ sở trí tuệ nhân tạo

Tên đồ án: Futoshiki (Giải bài toán bằng Logic vị từ bậc nhất và các thuật toán suy diễn)

Hình thức: Bài tập nhóm

Giảng viên: thầy Lê Ngọc Thành, thầy Huỳnh Lâm Hải Đăng, thầy Lê Nhựt Nam

Sinh viên thực hiện:

Nguyễn Nhựt Huy	24127398
Trần Quang Minh Huy	24127403
Trịnh Khánh Linh	24127437
Nguyễn Minh Khôi	24127066

Mục lục

Danh sách hình vẽ **1**

Danh sách bảng **2**

1 Phân công công việc **3**
1.1 Phân công chi tiết . . . 3
1.2 Yêu cầu đồ án . . . 4

2 Biểu diễn bài toán bằng Logic vị từ bậc nhất (FOL) **5**
2.1 Giới thiệu . . . 5
2.2 Từ vựng (Vocabulary) . . . 5
2.2.1 Miền giá trị . . . 5
2.2.2 Bảng vị từ . . . 5
2.3 Các tiên đề FOL . . . 5
2.4 Skolem hóa . . . 7
2.5 Chuyển đổi CNF . . . 7
2.5.1 A1 . . . 7
2.5.2 A3 . . . 7
2.5.3 A4 . . . 7
2.6 Kết luận . . . 8

3 Các thuật toán suy diễn **9**
3.1 Forward Chaining . . . 9
3.1.1 Xây dựng cơ sở tri thức (Knowledge Base) . . . 9
3.1.2 Thuật toán suy diễn . . . 9
3.1.3 Kết hợp Backtracking và heuristic MRV . . . 10
3.1.4 Ví dụ minh họa . . . 10
3.1.5 Nhận xét . . . 11
3.2 Backward Chaining . . . 12
3.3 Thuật toán A* . . . 12

4 Phân tích heuristic (A*) **13**

5 Kết quả thực nghiệm **14**

6 So sánh và đánh giá **15**

Danh sách hình vẽ

Danh sách bảng

1.1 Phân công công việc các thành viên 3
1.2 Bảng tự đánh giá yêu cầu bắt buộc 4

Phần 1
Phân công công việc

1.1 Phân công chi tiết

Bảng 1.1: Phân công công việc các thành viên

Thành viên	**Công việc được giao**	**Hoàn thành**
Nguyễn Nhựt Huy	• Quản lý repository nhóm	100%
Trần Quang Minh Huy	• Cài đặt thuật toán A* + heuristic	100%
Trịnh Khánh Linh	• Biểu diễn FOL + Skolem + CNF	100%
Nguyễn Minh Khôi	• Cài đặt thuật toán Brute-force + Backtracking	100%

1.2 Yêu cầu đồ án

Bảng 1.2: Bảng tự đánh giá yêu cầu bắt buộc

STT	Yêu cầu	Trạng thái	Ghi chú
1	Hình thức hóa FOL đầy đủ		Dạng Skolem & CNF cho $\geq$ 3 tiên đề
2	Sinh ground KB tự động theo N		Forward chaining, backward chaining, A*
3	Implement Forward Chaining		Viết từ đầu, đúng trên mọi test case
4	Implement Backward Chaining		Kiểu Prolog, truy vấn Val(i,j,?)
5	Implement A* Search		Heuristic chấp nhận được, có phân tích & so sánh
6	Implement Brute-force & Backtracking		Dùng để so sánh hiệu năng
7	Thiết kế $\geq$ 10 file input		Đa dạng kích thước: 4×4, 5×5, 6×6, 7×7, 9×9
8	Báo cáo đầy đủ		FOL, pseudocode, thực nghiệm (bảng + biểu đồ), so sánh thuật toán
9	Video demo		$\geq$ 3 test case, upload YouTube/Drive
10	Nộp đúng cấu trúc .zip		Docs/, Source/, README.txt, requirements.txt

Phần 2
Biểu diễn bài toán bằng Logic vị từ bậc nhất (FOL)

2.1 Giới thiệu

Futoshiki là một trò chơi giải đố logic được thực hiện trên một bảng kích thước $N \times N$. Nhiệm vụ của người chơi là điền các số từ 1 đến N vào các ô trống sao cho tất cả các ràng buộc của bài toán đều được thỏa mãn. Nhờ cấu trúc rõ ràng và các luật chơi mang tính logic chặt chẽ, Futoshiki thường được sử dụng như một ví dụ điển hình để áp dụng Lôgic Vị từ Bậc Nhất (FOL), vì các ràng buộc này có thể biểu diễn khá trực quan thông qua các mệnh đề có lượng từ.

Trong phần này, báo cáo sẽ trình bày cách xây dựng hệ tiên đề FOL cho bài toán Futoshiki kích thước 4×4. Nội dung bao gồm việc xác định từ vựng sử dụng, xây dựng các tiên đề tương ứng với luật chơi, đồng thời thực hiện Skolem hóa và chuyển đổi một số tiên đề tiêu biểu sang dạng chuẩn CNF.

2.2 Từ vựng (Vocabulary)

2.2.1 Miền giá trị

- Chỉ số hàng, cột: $i, j \in \{1, 2, 3, 4\}$
- Giá trị ô: $v \in \{1, 2, 3, 4\}$

2.2.2 Bảng vị từ

Vị từ	Bậc	Ý nghĩa
$Val(i, j, v)$	3	Ô (i, j) được gán giá trị v
$Given(i, j, v)$	3	Ô (i, j) có giá trị cho sẵn v
$LessH(i, j)$	2	Ràng buộc $<$ ngang giữa (i, j) và $(i, j+1)$
$GreaterH(i, j)$	2	Ràng buộc $>$ ngang giữa (i, j) và $(i, j+1)$
$LessV(i, j)$	2	Ràng buộc $<$ dọc giữa (i, j) và $(i+1, j)$
$GreaterV(i, j)$	2	Ràng buộc $>$ dọc giữa (i, j) và $(i+1, j)$
$Less(v1, v2)$	2	Quan hệ số học $v_1 < v_2$

2.3 Các tiên đề FOL

A1 - Mỗi ô có ít nhất một giá trị

Luật gốc: Mỗi ô trong bảng đều phải được điền một giá trị từ 1 → 4.

$$\forall i \forall j \exists v \; Val(i, j, v)$$

A2 - Mỗi ô có nhiều nhất một giá trị

Luật gốc: Mỗi ô không thể chứa đồng thời hai giá trị.

$$\forall i \forall j \forall v_1 \forall v_2 \ (Val(i,j,v_1) \wedge Val(i,j,v_2)) \Rightarrow v_1 = v_2$$

A3 - Không trùng giá trị trong hàng

Luật gốc: Mỗi hàng là một hoán vị của $\{1,2,3,4\}$, tức là không có giá trị nào trong hàng trùng nhau.

$$\forall i \forall j_1 \forall j_2 \forall v \ (Val(i,j_1,v) \wedge Val(i,j_2,v) \wedge j_1 \neq j_2) \Rightarrow \perp$$

A3′ - Không trùng giá trị trong cột

Luật gốc: Mỗi cột là một hoán vị của $\{1,2,3,4\}$, tức là không có giá trị nào trong cột trùng nhau.

$$\forall j \forall i_1 \forall i_2 \forall v \ (Val(i_1,j,v) \wedge Val(i_2,j,v) \wedge i_1 \neq i_2) \Rightarrow \perp$$

A4 - Bất đẳng thức ngang $<$

Luật gốc: Nếu có dấu "$<$"giữa hai ô (i,j) và $(i,j+1)$ thì giá trị ô trái nhỏ hơn ô phải.

$$\forall i \forall j \forall v_1 \forall v_2 \ (LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2)) \Rightarrow Less(v_1,v_2)$$

A4′ - Bất đẳng thức ngang $>$

Luật gốc: Nếu có dấu "$>$"giữa hai ô (i,j) và $(i,j+1)$ thì giá trị ô phải nhỏ hơn ô trái.

$$\forall i \forall j \forall v_1 \forall v_2 \ (GreaterH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2)) \Rightarrow Less(v_2,v_1)$$

A5 - Bất đẳng thức dọc $<$

Luật gốc: Nếu có dấu "$<$"giữa hai ô (i,j) và $(i+1,j)$ thì giá trị ô trên nhỏ hơn ô dưới.

$$\forall i \forall j \forall v_1 \forall v_2 \ (LessV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2)) \Rightarrow Less(v_1,v_2)$$

A5′ - Bất đẳng thức dọc $>$

Luật gốc: Nếu có dấu "$>$"giữa hai ô (i,j) và $(i+1,j)$ thì giá trị ô dưới nhỏ hơn ô trên.

$$\forall i \forall j \forall v_1 \forall v_2 \ (GreaterV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2)) \Rightarrow Less(v_2,v_1)$$

A6 - Ô cho sẵn

Luật gốc: Các ô được điền sẵn phải giữ nguyên giá trị đó.

$$\forall i \forall j \forall v \ Given(i,j,v) \Rightarrow Val(i,j,v)$$

A7 - Ràng buộc miền giá trị

Luật gốc: Chỉ được điền các số thuộc $\{1, 2, 3, 4\}$*, không được điền số ngoài miền giá trị*

$$\forall i \forall j \forall v \; Val(i, j, v) \Rightarrow (v = 1 \vee v = 2 \vee v = 3 \vee v = 4)$$

2.4 Skolem hóa

Chỉ có A1 chứa lượng từ tồn tại:

$$\forall i \forall j \exists v \; Val(i, j, v)$$

Phân tích cấu trúc lượng từ:

- $\exists v$ nằm trong phạm vi của $\forall i$ và $\forall j$.
- Vì vậy ta thay v bằng hàm Skolem $f(i, j)$ - hàm hai biến phụ thuộc vào i và j.
- Loại bỏ $\exists v$, giữ nguyên $\forall i \forall j$

Sau Skolem hóa:

$$\forall i \forall j \; Val(i, j, f(i, j))$$

$\Rightarrow$ Với mỗi cặp tọa độ (i, j) cụ thể, hàm $f(i, j)$ trả về một giá trị xác định được gán cho ô đó. Tức là, ta không cần biết cụ thể giá trị là gì - chỉ cần biết nó tồn tại và duy nhất ứng với mỗi $\forall i \forall j$.

2.5 Chuyển đổi CNF

2.5.1 A1

$$\forall i \forall j \; Val(i, j, f(i, j))$$

$$Val(i, j, f(i, j))$$

2.5.2 A3

$$\forall i \forall j_1 \forall j_2 \forall v \; (Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2) \Rightarrow \perp$$

$$\forall i \forall j_1 \forall j_2 \forall v \; \neg[(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2)]$$

$$\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v) \vee (j_1 = j_2)$$

2.5.3 A4

$$\forall i \forall j \forall v_1 \forall v_2 \; (LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2)) \Rightarrow Less(v_1, v_2)$$

$$\forall i \forall j \forall v_1 \forall v_2 \; \neg[LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2)] \vee Less(v_1, v_2)$$

$$\neg LessH(i, j) \vee \neg Val(i, j, v_1) \vee \neg Val(i, j + 1, v_2) \vee Less(v_1, v_2)$$

2.6 Kết luận

Hệ tiên đề FOL gồm 10 tiên đề mô hình hóa đầy đủ luật chơi Futoshiki. Chỉ có một tiên đề cần Skolem hóa (A1), các tiên đề còn lại thuần phổ dụng. Sau khi chuyển sang CNF, hệ có thể sử dụng cho SAT solver hoặc suy diễn logic.

Phần 3
Các thuật toán suy diễn

3.1 Forward Chaining

3.1.1 Xây dựng cơ sở tri thức (Knowledge Base)

Cơ sở tri thức (KB) trong bài toán gồm hai thành phần chính:

- **Facts (Sự thật):** Là các thông tin đúng ngay từ đầu, ví dụ như các ô đã cho sẵn giá trị hoặc các ràng buộc bất đẳng thức trên bảng.
- **Rules (Luật):** Có dạng:
$$p_1 \wedge p_2 \wedge \cdots \wedge p_n \Rightarrow q$$
trong đó các p_i là điều kiện và q là kết luận.

Ngoài ra, mỗi luật được gắn thêm một biến đếm `count` để theo dõi số lượng điều kiện chưa được thỏa mãn, giúp tối ưu quá trình suy diễn.

Từ dữ liệu đầu vào của bảng $N \times N$, nhóm tự động sinh ra KB dựa trên các loại luật sau:

- **Dữ kiện ban đầu:** Thêm các fact như `Given(i,j,v)` cho các ô đã có sẵn giá trị, và các ràng buộc như `LessH`, `GreaterV`, ...
- **Luật gán giá trị:**
$$Given(i, j, v) \Rightarrow Val(i, j, v)$$
- **Luật ràng buộc hàng và cột:** Nếu một ô đã nhận giá trị v thì các ô khác cùng hàng/cột không được nhận giá trị đó.
- **Luật bất đẳng thức:** Dựa vào các dấu $<, >$ trên bảng để loại trừ các giá trị không hợp lệ của các ô liên quan.

3.1.2 Thuật toán suy diễn

Thuật toán được cài đặt theo hướng sử dụng **hàng đợi (agenda)** để quản lý các fact cần xử lý. Đồng thời, nhóm sử dụng thêm một cấu trúc chỉ mục (tương tự hash map) để nhanh chóng xác định các luật bị ảnh hưởng khi một fact mới xuất hiện.

Ý tưởng chính của Forward Chaining là lặp lại quá trình suy diễn cho đến khi đạt trạng thái ổn định (không sinh thêm fact mới). Cụ thể:

- Khởi tạo agenda với các fact ban đầu.
- Lặp lại:
  - Lấy một fact từ agenda.
  - Cập nhật các luật liên quan (giảm `count`).
  - Nếu một luật thỏa mãn (`count` = 0), suy ra fact mới.
  - Thêm fact mới vào KB và đưa vào agenda.

- Dừng khi agenda rỗng (đạt điểm cố định).

Sau bước này:

- Nếu tất cả các ô đã được xác định giá trị → kết thúc.
- Ngược lại → chuyển sang Backtracking để tiếp tục tìm nghiệm.

3.1.3 Kết hợp Backtracking và heuristic MRV

Khi Forward Chaining không thể suy diễn thêm (đạt trạng thái ổn định) nhưng bài toán vẫn chưa giải xong, nhóm sử dụng **Backtracking** để tìm kiếm phần còn lại của nghiệm.

Để giảm không gian tìm kiếm, thuật toán sử dụng heuristic **MRV (Minimum Remaining Values)**. Thay vì chọn ô theo thứ tự cố định, MRV ưu tiên chọn ô có domain nhỏ nhất (tức là ít giá trị khả thi nhất).

Ý tưởng của MRV:

- Ô có domain nhỏ thường bị ràng buộc nhiều hơn.
- Dễ phát hiện mâu thuẫn sớm nếu chọn sai.
- Giảm số nhánh cần duyệt trong cây tìm kiếm.

Quy trình Backtracking kết hợp MRV như sau:

- Chọn ô (i, j) chưa xác định với domain nhỏ nhất.
- Thử lần lượt từng giá trị v trong domain(i, j):
  - Gán $(i, j) = v$ và thêm fact `Val(i,j,v)` vào KB.
  - Chạy lại Forward Chaining để lan truyền ràng buộc.
  - Nếu xảy ra mâu thuẫn (domain rỗng hoặc conflict) thì quay lui và thử giá trị khác.

Nhờ kết hợp Forward Chaining và MRV, thuật toán vừa tận dụng được khả năng suy diễn để thu hẹp domain, vừa đảm bảo tìm được nghiệm đầy đủ với số bước thử ít hơn so với Backtracking thuần túy.

3.1.4 Ví dụ minh họa

Xét bài toán Futoshiki kích thước 3×3 với dữ liệu đầu vào như sau (intput1):

Grid:

$$\begin{bmatrix} 0 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Ràng buộc ngang:

- $(1, 1) < (1, 2)$

Ràng buộc dọc:

- $(1, 2) > (2, 2)$
- $(2, 1) > (3, 1)$

Bước 1: Khởi tạo KB và Domain

- Từ dữ kiện ban đầu: `Given(1,2,2)` $\Rightarrow$ `Val(1,2,2)`
- Domain ban đầu:

$$(1, 2) = \{2\}, \quad \text{các ô còn lại} = \{1, 2, 3\}$$

Bước 2: Forward Chaining

Từ `Val(1,2,2)`, suy ra:

- Loại bỏ 2 khỏi hàng 1: $(1,1), (1,3)$ không thể là 2
- Loại bỏ 2 khỏi cột 2: $(2,2), (3,2)$ không thể là 2

Xét ràng buộc $(1,1) < (1,2)$ với $(1,2) = 2$:

- $(1,1) < 2 \Rightarrow (1,1) = 1$

Suy ra:

`Val(1,1,1)`

Tiếp tục Forward Chaining:

- Loại 1 khỏi hàng 1 và cột 1

Sau một số bước suy diễn, domain được thu hẹp nhưng vẫn chưa xác định hết các ô.

Bước 3: Áp dụng MRV

Xét các ô chưa xác định, chọn ô có domain nhỏ nhất. Ví dụ:

- Giả sử ô $(2,2)$ còn domain $\{1,3\}$ (nhỏ nhất)

Chọn ô $(2,2)$ để thử giá trị.

Bước 4: Backtracking + Forward Chaining

- Thử $(2,2) = 1$:
  - Thêm `Val(2,2,1)` vào KB
  - Chạy lại Forward Chaining
  - Nếu xảy ra mâu thuẫn $\rightarrow$ quay lui
- Nếu không hợp lệ, thử $(2,2) = 3$:
  - Tiếp tục suy diễn và lan truyền ràng buộc

Quá trình này tiếp tục cho đến khi tất cả các ô đều được gán giá trị.

Nhận xét:

Ví dụ trên cho thấy:

- Forward Chaining giúp xác định nhanh các ô như $(1,1)$ mà không cần thử.
- MRV giúp chọn đúng ô "khó" để thử trước, giảm số nhánh cần duyệt.
- Sự kết hợp hai phương pháp giúp giảm đáng kể không gian tìm kiếm so với Backtracking thuần túy.
- Ban đầu, đưa các fact `Given(1,1,1)` và ràng buộc vào hàng đợi.
- Từ `Given(1,1,1)` suy ra `Val(1,1,1)`.
- Từ `Val(1,1,1)`, áp dụng các luật ràng buộc để loại bỏ giá trị 1 khỏi các ô cùng hàng và cột.
- Các fact dạng `NotVal` tiếp tục kích hoạt các luật khác, dần dần thu hẹp miền giá trị của các ô còn lại.
- Cuối cùng, khi mỗi ô chỉ còn một giá trị hợp lệ, ta thu được nghiệm hoàn chỉnh của bảng.

3.1.5 Nhận xét

Forward Chaining giúp lan truyền ràng buộc rất nhanh và giảm đáng kể không gian tìm kiếm ban đầu. Tuy nhiên, do chỉ dựa vào suy diễn nên thuật toán có thể bị dừng sớm khi không còn luật nào áp dụng được.

Vì vậy, việc kết hợp với Backtracking là cần thiết để đảm bảo tìm được nghiệm đầy đủ cho mọi test case.

3.2 Backward Chaining

3.3 Thuật toán A*

Phần 4
Phân tích heuristic (A*)

Phần 5
Kết quả thực nghiệm

Phần 6
So sánh và đánh giá